

Software Design Document

Connect Vibe



Submitted by

M. Tayyab

S23BINFT1M01107

Submitted to

Sir Ali Nawaz

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Summary

ConnectVibe is a web based video sharing platform designed to allow users to upload, watch, and interact with videos through a clean and user friendly interface. The system enables users to register accounts, log in securely, upload video content with titles and descriptions, interact with videos through likes and comments, manage their personal profiles, and search for content using keywords. The system also includes a dedicated admin panel where administrators can monitor, manage, and moderate all users and uploaded content across the platform.

ConnectVibe is built on the MVC (Model View Controller) architecture using Django as the backend framework and MySQL as the relational database management system. The frontend is developed using HTML, CSS, and JavaScript to ensure responsiveness and accessibility across standard web browsers. The platform is designed with a focus on performance, security, usability, and scalability so that it can handle growing user traffic and be extended with additional features in future versions. Overall, ConnectVibe delivers a complete, efficient, and reliable environment for video sharing and user engagement.

Contents

1. Application Architecture	5
1.1 MVC Architecture	5
1.2 Proposed Models	6
1.3 Proposed Views	7
1.4 Proposed Controllers	7
1.4.1 User Authentication Controller.....	7
1.4.2 Video Management Controller.....	8
1.4.3 Interaction Controller	8
1.4.4 Admin Controller	9
1.5 APIs and External Libraries	9
1.6 UML Diagrams	10
1.7 Coding Conventions	10
2. Data Model Schema	11
2.1 Entities	11
2.1.1 User Entity.....	11
2.1.2 Video Entity	12
2.1.3 Comment Entity	13
2.1.4 Like Entity	13
2.1.5 Admin Entity	14
2.1.6 Entity Relationships	14
2.2 Database Constraints	15
2.3 Future Database Enhancements	16
3. User Interface	16
3.1 User Interface Design Philosophy	17
3.2 Color Scheme	17
3.3 Typography	18
3.4 Responsiveness and Screen Constraints	19
3.5 User Interface Screens	19
3.5.1 Registration Page	20
3.5.2 Login Page	20
3.5.3 Home Feed Page	21
3.5.4 Video Upload Page	22

3.5.5 Profile Page.....	23
3.5.6 Search Results Page	24
3.5.7 Admin Panel Page	25
3.6 Future UI Enhancements.....	27
4. References	27

1. Application Architecture

ConnectVibe is developed using the MVC (Model View Controller) architectural pattern, which is a widely adopted and industry standard approach to building scalable, maintainable, and well structured web applications. This architecture cleanly separates the concerns of data management, business logic, and user interface into three distinct layers, making the system easier to develop, test, and maintain over time.

The overall system operates on a client server model. The client is a standard web browser from which users access the ConnectVibe platform. The server is the Django based backend application that processes all incoming requests, applies business logic, communicates with the MySQL database, and returns the appropriate responses to the client. All communication between client and server occurs over the HTTP/HTTPS protocol.

The system is divided into three primary tiers. The first is the Presentation Tier, which is everything the user sees and interacts with in the browser. The second is the Application Logic Tier, which is the Django backend responsible for processing requests, executing business rules, and serving data. The third is the Data Tier, which is the MySQL database that stores and manages all persistent information including users, videos, comments, likes, and logs.

1.1 MVC Architecture

Component of MVC

Model :

Handles all database operations and defines the data structure of the application

View :

Manages the display and rendering of the graphical user interface presented to users

Controller :

Processes incoming HTTP requests, applies business logic, and coordinates between Model and

View :

The Model layer in ConnectVibe represents all data entities such as User, Video, Comment, Like, and Admin. Each model maps directly to a table in the MySQL database. Django's ORM (Object Relational Mapper) is used to interact with these models without writing raw SQL queries, improving both security and development speed.

The View layer consists of Django HTML templates rendered on the server side. Each view template corresponds to a specific page of the application such as the home feed, video player, profile page, or admin panel. Templates use Django's templating engine to inject dynamic data received from the controllers.

The Controller layer in Django is represented by views.py files and URL routing configurations. Each controller function receives an HTTP request, processes it using the relevant model, and returns a rendered HTML response or a redirect to another page.

1.2 Proposed Models

The following models will be implemented in the ConnectVibe system:

- User
- Video
- Comment
- Like
- Admin

Each model is defined as a Python class inheriting from Django's base Model class. Fields, data types, constraints, and relationships between models are declared within these classes, and Django's migration system automatically generates and applies the corresponding database schema.

1.3 Proposed Views

The following views will be implemented in the ConnectVibe system:

Views

- Registration View
- Login View
- Home Feed View
- Video Player View
- Video Upload View
- Profile View
- Search Results View
- Admin Dashboard View
- Admin User Management View
- Admin Video Management View

Each view is a Django HTML template file that receives a context dictionary of data from its corresponding controller and renders it into dynamic HTML. Views are designed to be clean, minimal, and responsive.

1.4 Proposed Controllers

Controllers are the core logic units of the ConnectVibe application. Each controller is a set of Python functions defined in Django's `views.py` file that handle specific groups of related HTTP requests. Controllers receive requests from the browser, interact with the relevant models to fetch or modify data, and return a rendered view or a JSON response. Below are the proposed controllers and their functions.

1.4.1 User Authentication Controller

This controller manages all user account related operations including registration, login, logout, and session handling. It validates form data, hashes passwords before storage, manages session tokens, and enforces authentication requirements on protected routes.

Functions:

- register_user()
- login_user()
- logout_user()
- validate_session()
- update_profile()

1.4.2 Video Management Controller

This controller handles all video related operations including video upload, metadata storage, video retrieval for playback, and video deletion. It manages file handling for video uploads, stores file paths in the database, and increments view counts on each video playback request.

Functions:

upload_video()
get_video()
delete_video()
increment_view_count()
get_user_videos()

1.4.3 Interaction Controller

This controller manages user engagement features including commenting on videos and liking or unliking videos. It enforces the one like per user rule per video and handles comment submission, retrieval, and deletion.

Functions:

- add_comment()
- delete_comment()
- toggle_like()
- get_comments()
- get_like_count()

1.4.4 Admin Controller

This controller manages all administrative operations available through the admin panel. It enforces admin only access using role based authorization checks and provides functionality to view, manage, and delete users and video content.

Functions:

- `get_all_users()`
- `delete_user()`
- `get_all_videos()`
- `delete_video_admin()`
- `get_system_stats()`

1.5 APIs and External Libraries

API / Library Purpose

Django :

Backend framework of the project. Provides the MVC structure, ORM, URL routing, templating engine, form validation, and session management so development can focus on business logic.

MySQL Connector / mysqlclient :

Enables Django to connect and communicate with the MySQL relational database.

Pillow :

Python imaging library used to handle profile picture uploads, resize images, and validate image formats before storage.

Django Authentication Module : Built in Django module that provides secure password hashing, session management, login/logout utilities, and access control decorators.

HTML / CSS / JavaScript : Frontend technologies used to build and style all user interface pages rendered in the browser.

Bootstrap : Frontend CSS framework used to provide responsive grid layouts, pre built UI components, and consistent styling across all pages.

1.6 UML Diagrams

The following diagrams describe the structural and behavioral design of the ConnectVibe system:

Use Case Diagram : illustrates all interactions between the User and Admin actors and the system's functional features.

Sequence Diagram : shows the step by step flow of messages and operations during key processes such as video upload and user login.

ER Diagram : defines the relationships between all database entities including User, Video, Comment, Like, and Admin.

Data Flow Diagram (DFD) : illustrates how data moves through the system from user input to backend processing to database storage and back to the user.

Class Diagram : represents the structure of each model class, its attributes, data types, and associations with other classes.

These diagrams are referenced and explained in detail in the accompanying SRS document submitted for ConnectVibe.

1.7 Coding Conventions

All code written for ConnectVibe follows standardized conventions to ensure readability, consistency, and ease of maintenance across the entire codebase.

Class Names : PascalCase (e.g., VideoUpload, UserProfile)

Variables : snake_case (e.g., video_title, user_id)

Functions : snake_case (e.g., get_video(), delete_user())

Constants : UPPER_CASE (e.g., MAX_FILE_SIZE, BASE_URL)

HTML Templates : lowercase with hyphens (e.g., home_feed.html, video_player.html)

CSS Classes : lowercase with hyphens (e.g., video_card, profile_header)

Database Table Names : lowercase with underscores (e.g., video_table, comment_table)

All Python code follows PEP 8 style guidelines. Comments are written in English and placed above the code block they describe. Each controller function includes a brief docstring explaining its purpose, expected inputs, and return values. HTML templates are indented with two spaces for clarity.

2. Data Model Schema

The data model schema is the complete blueprint of the ConnectVibe database. It defines every table, every field within each table, the data types of those fields, the constraints applied to them, and the relationships between different tables. A well designed data model is essential to ensure data integrity, eliminate redundancy, support efficient querying, and allow the system to scale as the number of users and videos grows over time.

ConnectVibe uses a relational database schema implemented in MySQL. Django's ORM is used to define and interact with this schema at the Python level, while MySQL manages the actual storage and enforcement of constraints at the database level. The schema is designed around five core entities: User, Video, Comment, Like, and Admin.

2.1 Entities

Entities are the fundamental tables in the ConnectVibe database. Each entity represents a distinct real world object or concept that the system needs to store information about. Entities define what data is collected, what rules and data types apply to each piece of data, and how entities relate to one another. The following entities are used in the ConnectVibe system database.

2.1.1 User Entity

The User entity stores all account related information for every registered user on the platform. This is the central entity in the database, as most other entities reference it through foreign key relationships.

Attribute	Data Type	Constraints	Description
------------------	------------------	--------------------	--------------------

user_id	INT	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each user
username	VARCHAR(100)	NOT NULL, UNIQUE	Chosen display name of the user
email	VARCHAR(150)	NOT NULL, UNIQUE	Email address used for login and notifications
password_hash	VARCHAR(255)	NOT NULL	Hashed and salted password string
profile_picture	VARCHAR(255)	NULLABLE	File path to the user's uploaded profile image
bio	TEXT	NULLABLE	Short personal description written by the user
created_	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp recording when the account was created

2.1.2 Video Entity

The Video entity stores all metadata associated with each video uploaded to the platform. It references the User entity to record which user uploaded each video.

Attribute	Data Type	Constraints	Description
video_id	INT	PRIMARY KEY AUTO INCREMENT	Unique identifier for each video
user_id	INT	FOREIGN KEY → User.user_id	References the uploader's user account
title	VARCHAR(200)	NOT NULL	Title of the uploaded video
TEXT	NULLABLE		Optional description

			provided by the uploader
file_path	VARCHAR(255)	NOT NULL	Server side path to the stored video file
thumbnail_path	VARCHAR(255)	NULLABLE	Server side path to the video's thumbnail image
view_count	INT	DEFAULT 0	Total number of times the video has been viewed
uploaded_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp recording when the video was uploaded

2.1.3 Comment Entity

The Comment entity stores all comments submitted by users on videos. Each comment is linked to both the user who wrote it and the video it was posted on.

comment_id is an integer that serves as the primary key with auto-increment, uniquely identifying each comment in the table.

video_id is an integer foreign key referencing the Video table, linking each comment to the video it belongs to.

user_id is an integer foreign key referencing the User table, identifying the user who submitted the comment.

content is a non-nullable TEXT column that stores the full written body of the comment.

created_at is a DATETIME field that defaults to the current timestamp, automatically recording when the comment was posted.

2.1.4 Like Entity

The Like entity records each instance of a user liking a video. A composite unique constraint on video_id and user_id ensures that one user can only like a given video once.

like_id is an integer that serves as the primary key with auto-increment, uniquely identifying each like record in the table.

video_id is an integer foreign key referencing the Video table, linking each like to the specific video that was liked.

user_id is an integer foreign key referencing the User table, identifying the user who liked the video.

created_at is a DATETIME field that defaults to the current timestamp, automatically recording when the like was registered.

A UNIQUE constraint on (video_id, user_id) ensures that the same user cannot like the same video more than once, preventing duplicate like entries.

2.1.5 Admin Entity

The Admin entity records which user accounts have been granted administrative privileges. Rather than creating a completely separate user table for admins, this design links admin rights to existing user accounts using a foreign key relationship.

admin_id is an integer that serves as the primary key with auto-increment, uniquely identifying each admin record in the table.

user_id is an integer foreign key referencing the User table with a UNIQUE constraint, ensuring each user account can hold only one admin record.

role is a VARCHAR(50) field that defaults to 'admin', describing the administrative role assigned to the user.

granted_at is a DATETIME field that defaults to the current timestamp, automatically recording when the admin rights were granted.

2.1.6 Entity Relationships

Relationship	Type	Description
--------------	------	-------------

User → Video One to Many One user can upload many videos; each video belongs to one user

User → Comment One to Many One user can post many comments; each comment belongs to one user

User → Like One to Many One user can like many videos; each like record belongs to one user

Video → Comment One to Many One video can have many comments; each comment belongs to one video

Video → Like One to Many One video can receive many likes; each like record belongs to one video

User → Admin One to One One user account can be linked to at most one admin record

2.2 Database Constraints

Database constraints are rules enforced at the database level to maintain the correctness, consistency, and integrity of all data stored in the ConnectVibe MySQL database. These constraints ensure that invalid, incomplete, or contradictory data cannot be entered into the database regardless of how the application layer behaves.

Constraint Purpose

PRIMARY KEY Uniquely identifies each record in a table and prevents duplicate entries

FOREIGN KEY Maintains referential integrity between related tables by linking records correctly

NOT NULL Prevents critical fields from being left empty when a record is created

UNIQUE Prevents duplicate values in fields that must be distinct such as email and username

DEFAULT Automatically assigns a defined value to a field when no value is explicitly provided

CHECK Validates that data values fall within acceptable ranges or match expected formats

COMPOSITE UNIQUE Prevents duplicate combinations such as the same user liking the same video twice

ON DELETE CASCADE constraints are applied to foreign key relationships involving Comment and Like entities. This means that if a video is deleted, all comments and likes associated with that video are automatically deleted as well, preventing orphaned records from accumulating in the database.

2.3 Future Database Enhancements

The current ConnectVibe database schema is designed to support all core features defined in the SRS. However, as the platform grows and additional features are developed, the following database enhancements are planned for future versions:

Subscription / Follow Table to allow users to follow other users and receive updates about new uploads from accounts they follow.

Notification Table to store system generated and user triggered notifications for events such as new comments, likes, and uploads.

Video Category / Tag Table to allow videos to be tagged with topics or categories for better content discovery and filtering.

View History Table to record each user's video watch history, enabling personalized recommendations and a "watch again" feature.

Report Table to allow users to flag inappropriate videos or comments for admin review and moderation.

User Preferences Table to store per user settings such as notification preferences, language settings, and display preferences.

3. User Interface

The User Interface (UI) is the visual layer of the ConnectVibe platform through which all users directly interact with the system. A well designed UI is critical to the success of a video sharing platform because it determines how easily and intuitively users can find content, upload videos, engage with the community, and manage their accounts. ConnectVibe's UI is designed with a strong emphasis on clarity, accessibility, and modern aesthetic principles to ensure a positive experience for all users regardless of their level of technical knowledge.

The UI is built entirely using HTML, CSS, JavaScript, and the Bootstrap framework. All pages are server rendered by Django's templating engine, which injects dynamic data from the database into the HTML before delivering it to the browser.

3.1 User Interface Design Philosophy

ConnectVibe follows a modern, clean, and user centered design philosophy. Every design decision is guided by the principles of Human Computer Interaction (HCI) to ensure that the interface is intuitive, readable, and efficient. The platform prioritizes the needs of general internet users who may not be technically experienced but expect a smooth and visually pleasant experience.

The overall UI philosophy focuses on:

Minimal and distraction free design that keeps the user's attention on the content
Clear visual hierarchy that guides users naturally from one element to the next
Consistent layout and navigation patterns across all pages so users can predict where to find things
Fast loading pages that minimize time to interaction
Accessible design with adequate font sizes, color contrast, and clearly labeled interactive elements
friendly and informative error messages that guide users to correct their input rather than leaving them confused

3.2 Color Scheme

The color scheme of ConnectVibe is designed to feel modern, clean, and content focused. The palette uses neutral and deep tones to keep the interface unobtrusive while allowing video thumbnails and content to stand out visually as the primary element on every page.

Purpose

Color

Primary Color : Deep Slate Blue (1a1f36) – used for the navigation bar and main headers

Secondary Color : Light Gray (f4f6f8) – used for page backgrounds and card surfaces

Accent Color: Vibrant Red (e63946) – used for like buttons, active states, and call to action elements

Text Primary : Near Black (212529) – used for all main body text and headings

Text Secondary : Medium Gray (6c757d) – used for timestamps, metadata, and secondary labels

Success Messages : Green (28a745) – used for upload success and profile update confirmations

Error Messages : Red (dc3545) – used for login failures, validation errors, and deletion warnings

Button Hover : Slightly darkened shade of the primary button color to indicate interactivity

This color scheme ensures sufficient contrast between background and foreground elements to meet basic accessibility standards and provide comfortable reading conditions under both bright and dim ambient lighting.

3.3 Typography

ConnectVibe uses clean and highly readable fonts to ensure that all text content on the platform is easy to read at any size. Typography choices are made to convey professionalism while remaining approachable and friendly.

Element Font Style

Page Headings (H1) : Bold, 28px, Primary Text Color

Section Headings (H2, H3) : Semi Bold, 20–24px, Primary Text Color

Body Text : Regular, 16px, Primary Text Color

Video Titles on Cards : Semi Bold, 15px, Primary Text Color

Metadata (views, dates) : Regular, 13px, Secondary Text Color

Buttons : Medium weight, 14–16px, White or Accent Color

Input Field Labels : Regular, 14px, Secondary Text Color

Font Features:

High legibility at all supported screen sizes

Consistent line height of 1.6 for comfortable paragraph reading

Adequate letter spacing on buttons and headings for clarity

Professional appearance suitable for a content sharing platform used by a wide audience

3.4 Responsiveness and Screen Constraints

ConnectVibe is designed to function correctly and display properly across a range of screen sizes commonly used to access web applications. The Bootstrap grid system is used to create flexible and adaptive layouts that adjust automatically based on the available screen width.

Supported Devices:

- Desktop computers
- Laptops
- Tablets (landscape and portrait)

Responsive Features:

Fluid grid layouts that reorganize columns based on screen width breakpoints

Navigation bar collapses into a hamburger menu on smaller screens

Video card grids shift from four columns on desktop to two columns on tablet and one column on mobile

Font sizes and button sizes scale appropriately to remain usable at smaller resolutions

Forms and input fields expand to full width on narrow screens

Recommended Resolutions:

1920 × 1080 for desktop monitors

1366 × 768 for standard laptops

768 × 1024 for tablets

3.5 User Interface Screens

User Interface Screens are the individual pages that users navigate through when using the ConnectVibe platform. Each screen is designed to serve a specific purpose and includes only the elements necessary to accomplish that purpose efficiently. Below is a detailed description of each screen, its purpose, its layout, the interactions it supports, and any preconditions that must be met before a user can access it.

3.5.1 Registration Page

The Registration Page is the first entry point for new users to the ConnectVibe platform. It allows visitors to create a personal account by providing their basic details. This page is publicly accessible and does not require any prior authentication.

Layout: The page presents a centered registration card on a light gray background. The card contains the ConnectVibe logo at the top, a "Create Your Account" heading, and a form with four input fields: Full Name, Email Address, Password, and Confirm Password. Below the form is a "Register" primary action button in the accent color. A line of secondary text below the button reads "Already have an account?" with a hyperlink directing the user to the Login Page.

Interaction: The user fills in all four fields and clicks the "Register" button. The controller validates that the email is not already registered, checks that the two password fields match, hashes the password, creates a new User record in the database, and redirects the user to the Login Page with a success message. If any validation fails, inline error messages appear directly below the relevant field without reloading the page.

Pre Conditions:

The user must not already have a registered account with the same email address.

The user must have an active internet connection.

Both password fields must contain matching values of at least eight characters.

3.5.2 Login Page

The Login Page allows registered users to authenticate themselves and gain access to the ConnectVibe platform. It is publicly accessible and serves as the gateway to all protected pages.

Layout:

The page uses the same centered card layout as the Registration Page for visual consistency. The card contains the ConnectVibe logo, a "Welcome Back" heading, and a form with two input fields: Email Address and Password. Below the form are a "Login" primary action button and a "Forgot Password?" secondary link. A "Don't have an account? Register" link is included below the button.

Interaction:

The user enters their registered email and password and clicks "Login." The controller queries the User table to find a matching email, verifies the submitted password against the stored hash using Django's password validation utilities, creates a session for the authenticated user, and redirects them to the Home Feed. If the credentials are incorrect, an error message is displayed above the form reading "Invalid email or password."

Pre Conditions:

The user must already have a registered and active account.

The user must enter the exact email and password used during registration.

3.5.3 Home Feed Page

The Home Feed Page is the main content discovery page of ConnectVibe. It is the first page a user sees after logging in and serves as the central hub from which all other sections of the platform are accessible.

Layout: The page consists of a fixed navigation bar at the top containing the ConnectVibe logo on the left, a centered search bar, and a user profile icon with a dropdown menu on the right. The dropdown menu provides links to the user's Profile Page, the Video Upload Page, and a Logout option. Below the navigation bar is the main content area, which displays a responsive grid of video cards. Each video card contains a thumbnail image occupying the upper two thirds of the card, the video title in semi bold text below the thumbnail, the uploader's username and profile picture in a small row, and the view count and upload date in secondary gray text at the bottom.

Interaction: The user scrolls through the grid to browse content. Clicking any video card navigates the user to that video's Video Player Page. Clicking the search bar and entering a keyword submits a search query and navigates to the Search Results Page.

Pre Conditions:

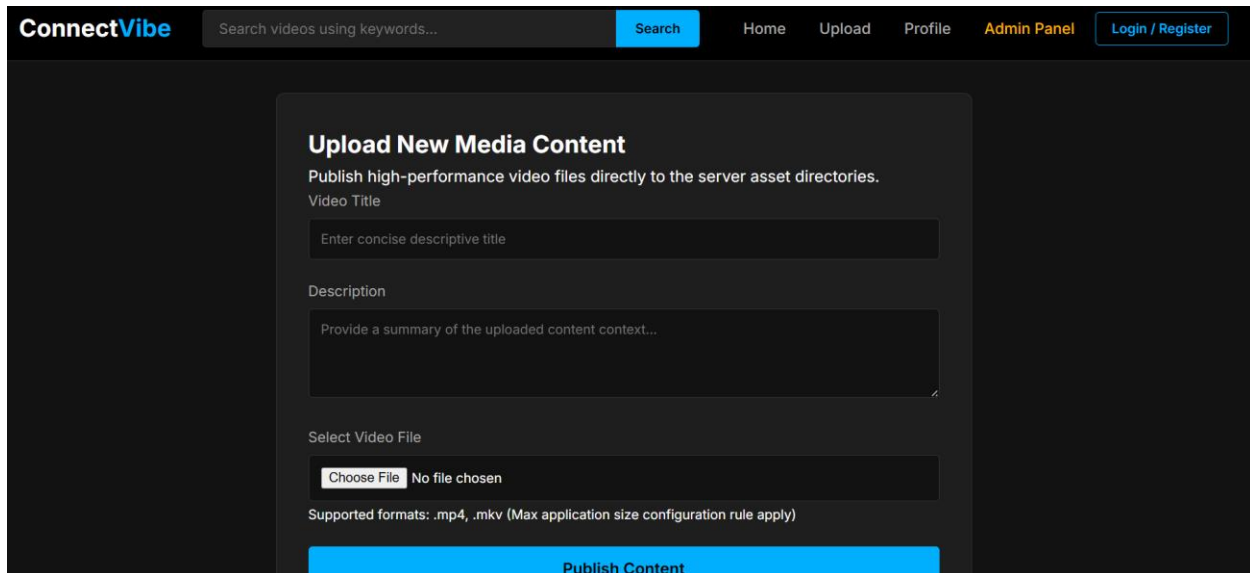
The user must be logged in with an active session.

At least one video must exist in the database for the feed to display content.

3.5.4 Video Upload Page

The Video Upload Page allows authenticated users to publish new video content to the ConnectVibe platform. Only logged in users may access this page.

Layout: The page contains a centered upload form card with the heading "Upload a New Video." The form includes a text input for the video title, a text area for the description, a file picker labeled "Choose Video File" that accepts common video formats such as MP4 and AVI, and an optional file picker labeled "Upload Thumbnail Image" that accepts image formats. A progress bar appears below the file pickers once the user begins uploading to show upload completion percentage. A "Publish Video" primary button is placed at the bottom of the form. A secondary "Cancel" button navigates back to the Home Feed without saving.

The screenshot shows the 'Upload New Media Content' form on the ConnectVibe website. The form is centered on a dark background. At the top, there's a navigation bar with the 'ConnectVibe' logo, a search bar, and links for 'Home', 'Upload', 'Profile', 'Admin Panel', and 'Login / Register'. The form itself has a title 'Upload New Media Content' and a subtitle 'Publish high-performance video files directly to the server asset directories.' It contains three main sections: 'Video Title' with a text input field, 'Description' with a text area, and 'Select Video File' with a file picker button labeled 'Choose File' and 'No file chosen'. Below the file picker, it lists 'Supported formats: .mp4, .mkv (Max application size configuration rule apply)'. At the bottom of the form is a large blue button labeled 'Publish Content'.

Interaction:

The user fills in the title, optionally the description, selects a video file, and optionally selects a thumbnail image. Clicking "Publish Video" submits the form. The controller validates that a title and video file have been provided, saves the video file to the server's media directory, stores the video metadata including the file path and uploader's user_id in the Video table, and redirects the user to the newly uploaded video's player page with a success message.

Pre Conditions:

The user must be logged in.

The uploaded file must be a valid video format.

The video title field must not be left empty.

3.5.5 Profile Page

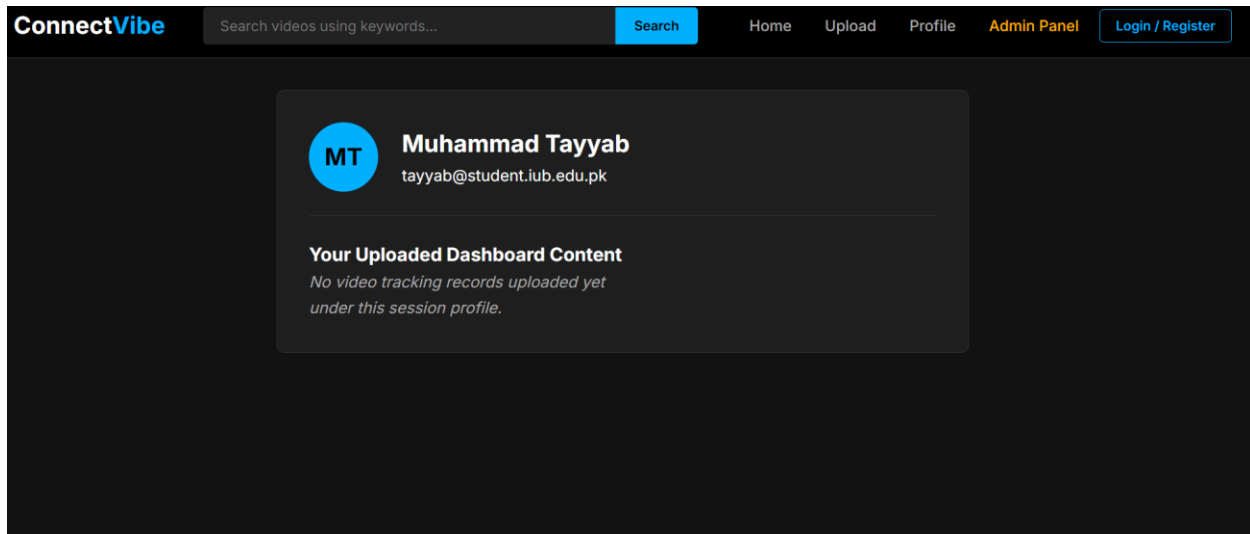
The Profile Page displays a user's public profile information and all of the videos they have uploaded to the platform. It can be viewed by any logged in user but can only be edited by the profile owner.

Layout:

The top of the page features a profile header section with a large circular profile picture on the left, the username in bold large text, the user's bio in regular text below the username, and the total number of uploaded videos displayed as a count. If the logged in user is viewing their own profile, an "Edit Profile" button appears in the top right corner of the header. Below the header is a grid of the user's uploaded videos using the same card layout as the Home Feed.

Interaction:

Clicking "Edit Profile" reveals an inline edit form directly below the header where the user can update their bio text and upload a new profile picture. Clicking "Save Changes" submits the updates to the User Management Controller, which updates the User record in the database and re-renders the profile page with the new information. Clicking any video card in the grid navigates to that video's Player Page.



Pre Conditions:

The user must be logged in to view any profile page.

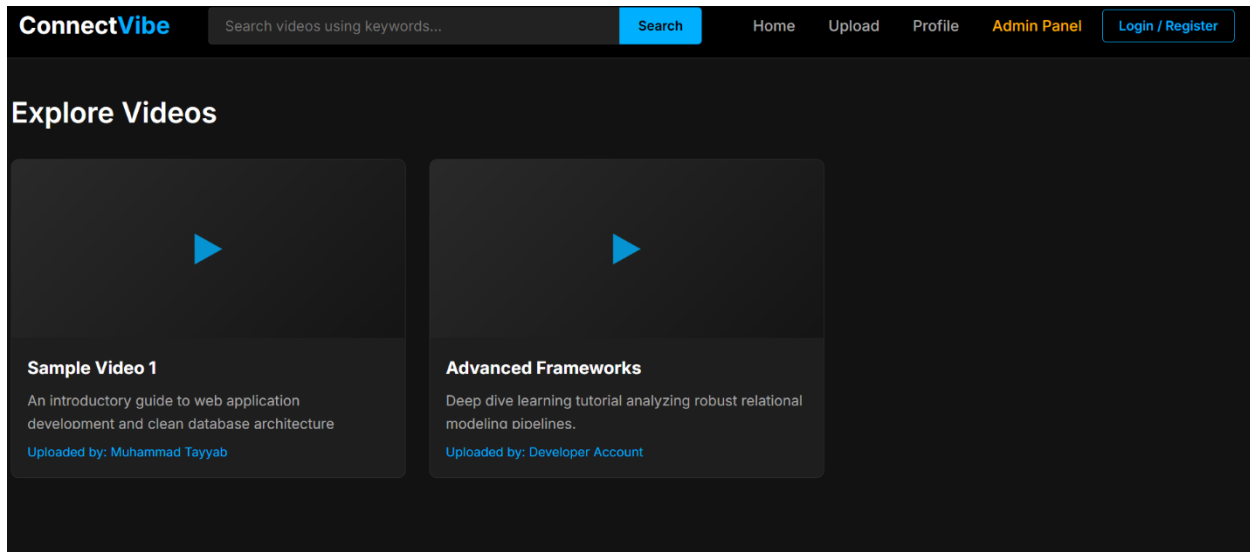
The profile being viewed must belong to a registered user.

3.5.6 Search Results Page

The Search Results Page displays all videos whose titles or descriptions match the keyword entered by the user in the navigation bar's search field.

Layout: The page contains a results header that reads "Search results for: [keyword]" in bold text, followed by the total number of results found in parentheses. Below the header is the same responsive video card grid layout used on the Home Feed. Each card shows the thumbnail, title, uploader name, and view count. If no videos match the search keyword, a centered message reads "No results found for '[keyword]'. Try a different search."

Interaction: Each video card is clickable and navigates to the Video Player Page for that video. The search bar in the navigation bar remains populated with the current keyword so users can easily modify and refine their search.



Pre Conditions:

The user must be logged in.

The user must have entered a non empty search keyword.

3.5.7 Admin Panel Page

The Admin Panel is a protected section of the ConnectVibe application accessible only to users whose accounts are linked to a record in the Admin entity. It provides administrators with tools to monitor and moderate all platform activity.

Layout: The Admin Panel uses a two column layout. The left column is a fixed sidebar containing navigation links to the following sections: Dashboard Overview, User Management, and Video Management. The right column displays the content area for whichever section is currently selected.

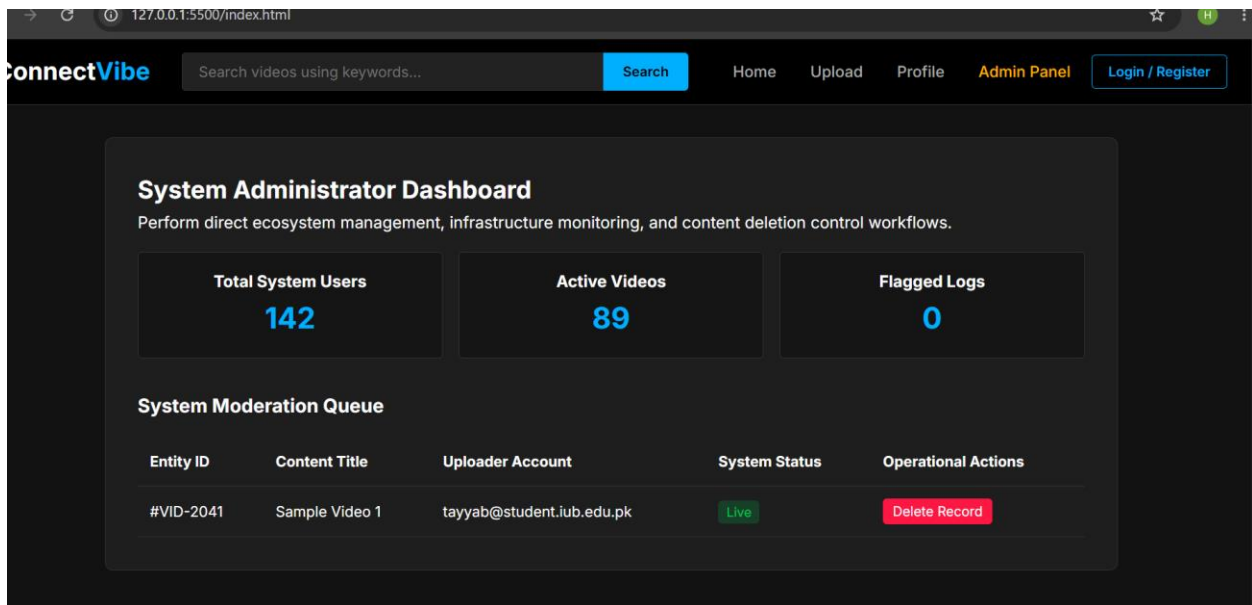
The Dashboard Overview section displays summary statistics including the total number of registered users, total number of uploaded videos, total number of comments, and total number of likes across the platform.

The User Management section displays a paginated data table listing all registered users. Each row shows the user's ID, username, email, and registration date. An action button in each row allows

the administrator to delete that user's account. Clicking the delete button opens a confirmation modal dialog asking the admin to confirm the action before it is executed.

The Video Management section displays a paginated data table listing all uploaded videos. Each row shows the video ID, title, uploader username, upload date, and view count. An action button in each row allows the administrator to delete any video. Clicking the delete button similarly opens a confirmation modal dialog before the deletion is processed.

Interaction: Clicking a delete button opens a modal dialog with a warning message and two buttons: "Confirm Delete" and "Cancel." Clicking "Confirm Delete" sends a POST request to the Admin Controller, which removes the relevant record from the database and refreshes the table. Cascade deletion rules ensure that deleting a user also removes all of their videos, comments, and likes.



Pre Conditions:

The user must be logged in and must have an admin record linked to their account.

Non admin users who attempt to access any admin URL are automatically redirected to the Home Feed with an "Access Denied" error message.

3.6 Future UI Enhancements

The current ConnectVibe user interface covers all core features defined in the SRS. As the platform evolves through future development cycles, the following UI enhancements are planned:

Dark Mode with a deep charcoal and silver color theme as an optional display setting controlled from user preferences.

Video Recommendation Sidebar showing suggested videos based on the currently playing video's topic or the user's watch history.

Infinite Scroll on the Home Feed and Search Results Page to replace pagination and load additional video cards automatically as the user reaches the bottom of the page.

Real time Comment Updates that display new comments posted by other users without requiring the page to be manually refreshed.

Notification Bell in the navigation bar that shows alerts for new likes and comments on the user's videos.

Mobile Application Interface extending the ConnectVibe experience to iOS and Android platforms with a native feeling responsive design.

Video Chapter Markers allowing uploaders to define timestamps with labels that appear on the player seek bar for navigation within longer videos.

These enhancements are not part of the initial release but will be introduced incrementally in future versioned updates following user feedback and platform growth.

4. References

[Django Software Foundation, "Django Documentation Version 4.2," 2023.](#)

[MySQL AB, "MySQL 8.0 Reference Manual," Oracle Corporation, 2023.](#)

[Bootstrap Team, "Bootstrap v5.3 Documentation," 2023.](#)

[Mozilla Developer Network, "HTTP Overview," MDN Web Docs, 2024.](#)

[W3Schools, "HTML, CSS, JavaScript Reference," 2024.](#)

[Python Software Foundation, "Python 3 Documentation," 2024.](#)

